

Implementation of a Local RAG System for Querying the Regulatory Code of the Municipality of Porto

A Semantic Retrieval Approach to Legal Document Question Answering

Rui Ribeiro

github.com/ruialexrib

August 8, 2026

Abstract

This paper presents the implementation of a local *Retrieval-Augmented Generation* (RAG) system for querying the Regulatory Code of the Municipality of Porto (CRMP) using natural language. The proposed solution combines document processing and segmentation, embedding generation, vector search, and a locally executed language model to generate responses grounded in the retrieved content, with references to the corresponding articles and pages. The system was integrated into a conversational application developed using n8n. In a preliminary evaluation with five questions, the relevant article was ranked first in all cases, resulting in *Hit@1*, *Recall@1*, and *MRR* scores of 1.0. These results demonstrate the technical feasibility of the proposed solution, although a more comprehensive evaluation is required to assess its robustness and overall quality.

Keywords: Information Retrieval; Regulatory Code; Embeddings; Vector Databases.

1 Introduction

The increasing digitalisation of legislation and public regulation has made large volumes of information available in electronic format. However, the consultation and interpretation of legal documents continue to present significant challenges.

Normative texts are characterised by rigid hierarchical structures and by relationships between different provisions, meaning that the interpretation of a legal rule may depend on its context and on other related provisions. This complexity may limit the effectiveness of traditional keyword-based search mechanisms, which do not always capture the semantic context required to identify legally relevant provisions [6].

Recent advances in *Large Language Models* (LLMs) have enabled the development of conversational interfaces capable of interpreting natural language questions and generating coherent responses. However, when used without external information retrieval mechanisms, these models may produce answers that are not grounded in the relevant documentary sources, including outdated, incorrect, or non-existent information. This limitation is particularly important in the legal domain, where accuracy and traceability are fundamental requirements [6].

The *Retrieval-Augmented Generation* (RAG) approach seeks to mitigate these limitations by combining information retrieval mechanisms with generative models, allowing knowledge from external sources to be incorporated into the generation process [3].

Rather than relying exclusively on the knowledge encoded in the model parameters during training, the system first retrieves the segments (*chunks*) of a corpus considered relevant to the question and provides them to the generative model as additional context for producing the answer [3]. This approach makes it possible to ground the generation process in retrieved

external information and to make the provenance of that information more explicit, although it does not eliminate the possibility of incorrect answers being generated.

In this context, this work describes the implementation of a local RAG system for querying the Regulatory Code of the Municipality of Porto (CRMP)¹ through questions formulated in natural language. The solution covers the entire processing pipeline, from the extraction and structuring of the original document to the provision of a conversational interface. The processing pipeline preserves elements of the legal structure, namely articles and pages, allowing the documentary provenance of the retrieved results to be maintained. In addition to implementing the solution, this work aims to document the different stages and technical decisions in order to provide a reference for its reproduction and a starting point for the evolution and experimentation of different retrieval strategies, models, and generation mechanisms.

This work has three main objectives: (i) to develop and document a reproducible RAG pipeline for the processing, indexing, and semantic retrieval of the CRMP; (ii) to integrate this pipeline into a conversational application capable of answering questions about the CRMP based on the retrieved context, while preserving references to the corresponding articles and source pages; and (iii) to perform a preliminary evaluation of the retrieval component using metrics such as *Hit@K*, *Recall@K*, and *Mean Reciprocal Rank* (MRR).

The architecture uses the `bge-m3`² model to generate vector representations of document segments and questions, Qdrant³ for vector storage and similarity search, and the AMALIA-9B language model [9], quantised and executed locally through Ollama⁴, for answer generation. The interaction between these components is subsequently integrated into an n8n⁵ workflow, which is responsible for orchestrating the query process.

The choice of a local architecture allows document processing, retrieval, and generation to be executed on local infrastructure, reducing dependence on external services and the operational costs associated with the use of commercial language model APIs.

The implementation is organised into two complementary repositories:

- `legal-rag-pt`, which contains the components responsible for document ingestion and structuring, content segmentation, embedding generation, Qdrant indexing, vector search, and evaluation of the retrieval component;
- `legal-rag-pt-n8n`, which contains the conversational application and the n8n workflow used to orchestrate question embedding generation, context retrieval, answer generation, and presentation of the corresponding sources.

The remainder of this paper is organised as follows. Section 2 presents the theoretical background, followed by the proposed architecture in Section 3. Section 4 describes the environment setup, while Section 5 details the corpus construction process. Section 6 presents embedding generation and indexing, and Section 7 describes the search and evaluation methodology. The conversational application implemented in n8n is presented in Section 8. Finally, Section 9 presents the conclusions and directions for future work.

2 Theoretical Background

This section presents the main concepts underlying the proposed solution, namely Large Language Models, vector representations of text, semantic retrieval, vector databases, and the *Retrieval-Augmented Generation* approach. The technologies used for local model execution and application orchestration are also introduced.

¹https://crmp.cm-porto.pt/apex/f?p=1501:5:0::NO::F100_ID_MENU:4581

²<https://huggingface.co/BAAI/bge-m3>

³<https://qdrant.tech/>

⁴<https://ollama.com/>

⁵<https://n8n.io/>

2.1 Large Language Models

Large Language Models (LLMs) are deep learning models trained on large volumes of text to learn patterns and representations of natural language. The Transformer architecture [10] underlies a large proportion of modern language models and introduced attention mechanisms that enable relationships between different elements of a sequence to be modelled.

The original Transformer architecture consists of two main components: the *encoder* and the *decoder*. The *encoder* transforms an input sequence into a contextual representation, while the *decoder* uses the available representations and previously generated elements to produce an output sequence. Different families of models have emerged from this architecture, including predominantly encoder-based models, which are well suited to text representation and understanding tasks, and decoder-based generative models, which are designed for sequential text generation.

This distinction is reflected in the two types of models used in this work. For vector representation of the content, `bge-m3` [1] is used as an *embedding* model that transforms questions and document segments into vector representations, allowing their semantic proximity to be subsequently compared. The model therefore performs an information encoding function rather than a text generation function.

For the generative component, AMALIA-9B [9], a language model developed for European Portuguese, is used. In this work, the model receives the question together with the previously retrieved documentary context and generates a natural language response based on that information.

These two types of models therefore perform distinct functions within the proposed architecture: `bge-m3` enables the representation and retrieval of semantically relevant information, whereas AMALIA-9B is responsible for generating the final response. This separation between retrieval and generation is a fundamental element of the *Retrieval-Augmented Generation* approach discussed in the following sections.

2.2 Embeddings

An *embedding* is a vector representation of an object, such as a word, sentence, or text segment, in a multidimensional space. These representations aim to encode semantic properties, enabling relationships between different textual elements to be represented computationally. Although these spaces typically have a large number of dimensions, Figure 1 presents a conceptual three-dimensional representation of this principle, in which semantically related elements tend to be located in nearby regions of the vector space.

Early widely adopted approaches relied on static *embeddings*, such as Word2Vec [5] and GloVe [7]. In these models, each word is associated with a single vector, regardless of the context in which it occurs. Consequently, a word with different meanings retains the same vector representation across all sentences, limiting the ability to represent phenomena such as polysemy.

The development of models based on the Transformer architecture enabled the use of contextual *embeddings*, in which the representation of a word depends on the other elements of the sequence in which it occurs. Models such as BERT [2] use attention mechanisms to produce contextualised representations, allowing the same word to have different representations depending on its linguistic context.

For information retrieval tasks, this evolution was subsequently extended to the representation of larger textual units, such as sentences, paragraphs, or documents. In these cases, a text segment can be represented by a single dense vector intended to capture its semantic content. Questions and document segments can therefore be projected into the same vector space and compared using a similarity measure.

This approach enables semantic retrieval and differs from methods based exclusively on lexical matching. Rather than searching only for occurrences of the same terms in the question and

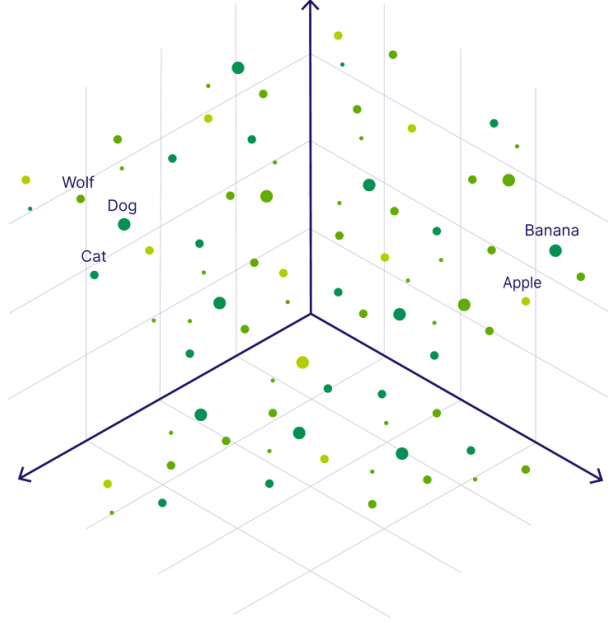


Figure 1: Conceptual representation of embeddings in a vector space. Semantically related elements tend to occupy nearby positions, as illustrated by the terms *Cat*, *Dog*, and *Wolf*, in contrast to *Apple* and *Banana*, which are located in a different region.

the document, vector retrieval aims to identify segments whose representations are close in the embedding space, allowing semantically related information to be retrieved even when different terminology is used.

In this work, the **bge-m3** model [1] is used to generate *embeddings* for both questions and CRMP segments. The same model is applied to both types of content, allowing them to be represented in a common vector space and their proximity to be compared.

2.3 Vector Databases

The use of *embeddings* in larger document collections requires appropriate mechanisms for storing, indexing, and searching vectors. Vector databases are systems designed to store vector representations and efficiently perform proximity searches, making it possible to identify elements whose representations are most similar to a given query vector.

In a vector search, the user’s question is first transformed into an *embedding* using the same model applied to the indexed documents. The resulting vector is then compared with the stored vectors, with proximity determined using a similarity or distance metric. Commonly used metrics include cosine similarity, inner product, and Euclidean distance.

Retrieval can be formulated as a k -Nearest Neighbors (k-NN) problem, in which the k vectors closest to the query vector are selected. However, exact comparison with all stored vectors becomes increasingly computationally demanding as the collection grows. For this reason, vector search systems frequently rely on *Approximate Nearest Neighbor* (ANN) algorithms, which aim to retrieve results close to the exact nearest neighbors with greater computational efficiency.

One approach used for this purpose is *Hierarchical Navigable Small World* (HNSW) [4], which organises vectors in a hierarchical graph structure to accelerate nearest-neighbor search. This approach is particularly suitable for searching high-dimensional vector spaces.

In this work, Qdrant is used as the vector database. For each CRMP segment, the corresponding *embedding* and associated document metadata are stored, including the text, article, heading, and source pages. During a query, the question *embedding* is used to search for the nearest vectors, retrieving the segments considered most relevant together with their associated

metadata.

2.4 Retrieval-Augmented Generation

The *Retrieval-Augmented Generation* (RAG) approach, proposed by Lewis et al. [3], combines information retrieval mechanisms with generative models. Its objective is to complement the knowledge acquired by the model during training with information obtained from external sources. For each question, the system identifies relevant information within a document collection and provides it to the model as additional context for answer generation.

The operation of a RAG system can be divided into two main stages: retrieval and generation. In the first stage, the question is used to identify the most relevant elements within a document collection. In the second stage, the retrieved information is provided to the language model, together with the original question, as context for generating the response.

The retrieval stage can rely on different strategies. Semantic retrieval uses *embeddings* to represent the question and documents in a vector space, enabling the identification of semantically similar content even when the same terms are not explicitly used. Alternatively, lexical retrieval methods, such as BM25 [8], determine relevance based on the occurrence and distribution of terms in the question and documents. These approaches have complementary characteristics: semantic search is suitable for identifying content with similar meaning, whereas lexical search can be particularly effective when specific terms require direct matching, such as names, references, or technical expressions.

Semantic and lexical retrieval can also be combined through a hybrid search strategy. In this case, the results obtained from both approaches are combined or reordered, aiming to benefit from both the semantic proximity provided by *embeddings* and the lexical matching provided by methods such as BM25.

In the context of this work, the external knowledge source corresponds to the Regulatory Code of the Municipality of Porto. The current implementation uses semantic retrieval: each question is converted into a vector representation using the `bge-m3` model and used to retrieve the CRMP segments considered most relevant. These segments, together with their associated metadata, are subsequently incorporated into the context provided to the AMALIA-9B model, which generates the final response.

2.5 Ollama

Language models can be executed through external services or deployed on local infrastructure. Local execution provides greater control over the models being used while reducing dependence on external services and the operational costs associated with commercial APIs.

Ollama is a platform that simplifies the local execution of language and *embedding* models, providing an interface for managing and using installed models. These models can be accessed programmatically through an API, facilitating their integration with other applications and services.

Ollama can be installed directly on major operating systems, including Windows and Linux, and can also be deployed using Docker containers⁶. In this work, Ollama provides the layer responsible for the local execution of the models used in the architecture, exposing them through an API that enables integration with the remaining components of the solution via HTTP requests.

2.6 n8n

n8n is a workflow orchestration platform based on a *low-code* approach. Through a node-based visual interface, it enables the construction of processing workflows and the integration of different

⁶<https://docs.ollama.com/docker>

systems, services, and APIs with a reduced need for custom code development. Each node represents a specific operation, such as receiving and transforming data, performing HTTP requests, or interacting with external services.

This approach facilitates the construction and modification of integration workflows while retaining the possibility of incorporating custom code or logic when required.

A RAG application involves the coordinated execution of several operations, from receiving the question and retrieving relevant information to building the context, invoking the language model, and presenting the final response. Representing these operations through a visual *workflow* makes the flow of information between the different components explicit and simplifies its configuration and subsequent evolution.

In this work, n8n is used as the orchestration layer of the conversational application, coordinating communication between the different components through their respective APIs. The *workflow* controls the flow of information from the initial question to the presentation of the final response.

3 Architecture

The solution architecture was designed in a modular manner, separating corpus processing and preparation from the operations performed at query time. This separation distinguishes tasks carried out in advance, such as document extraction and structuring, content segmentation, *embedding* generation, and vector index construction, from the operations that must be performed to answer each question.

Corpus preparation operations are performed only during the initial construction of the system or whenever the source document is updated. Consequently, computationally more demanding tasks, such as generating *embeddings* for all document segments, do not need to be repeated for each query. At query time, only the operations required to process the question, retrieve relevant information, and generate the corresponding response are performed.

The solution is therefore organised into two main phases: *ingestion and indexing*, responsible for preparing and indexing the document corpus, and *query and generation*, responsible for retrieving relevant information and generating a natural language response.

3.1 Ingestion and Indexing

The first phase is responsible for transforming the original document into a structured representation suitable for semantic retrieval. The process begins with the extraction of the CRMP content, provided in PDF format, using PyMuPDF⁷, a Python library for PDF document processing that enables the extraction of text and information associated with the document structure.

The extracted content is subsequently normalised and structured while preserving relevant elements of the document organisation, namely articles, their corresponding headings, and source pages. This information is retained throughout the subsequent stages, preserving the documentary provenance of the processed content.

After structuring, the articles are divided into *chunks*, i.e., controlled-size text segments that constitute the individual units used for indexing and retrieval. Each *chunk* retains the metadata required to identify and locate it within the source document.

For each *chunk*, the `bge-m3` model generates a vector representation (*embedding*). The resulting vector is stored in Qdrant together with the textual content and its associated document metadata, forming the vector collection subsequently used during the query process.

The result of this phase is therefore a vector index of the corpus that can be reused across successive queries. The complete extraction, structuring, segmentation, and *embedding* generation process only needs to be repeated when the source document is updated.

⁷<https://pymupdf.readthedocs.io/>

Figure 2 presents the overall ingestion and indexing workflow, from the original document to the storage of vector representations and their associated metadata in Qdrant.

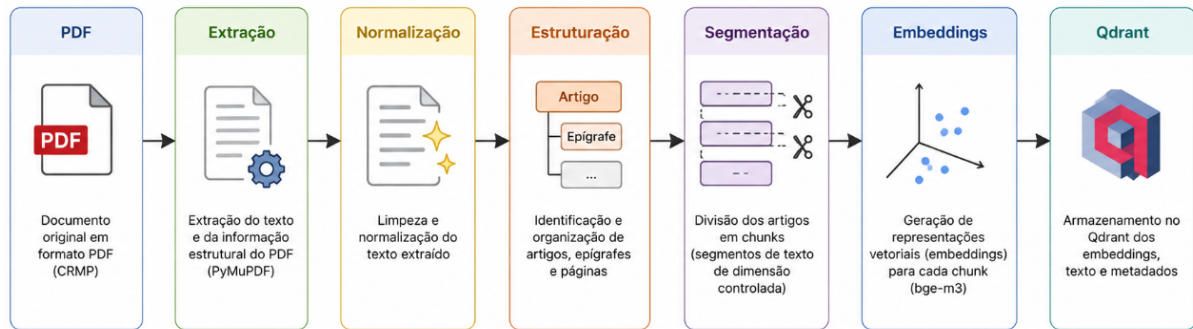


Figure 2: CRMP ingestion and indexing workflow. The original document is extracted and normalised, structured according to its document elements, segmented into *chunks*, and converted into vector representations using the *bge-m3* model. The *embeddings*, textual content, and associated metadata are subsequently stored in Qdrant.

3.2 Query and Generation

The second phase corresponds to the processing performed at query time and is executed whenever a question is submitted by the user. The *workflow* implemented in *n8n* receives the question and requests the corresponding *embedding* from the *bge-m3* model. The resulting vector is used to perform a similarity search in Qdrant, where it is compared with the vector representations of the previously indexed CRMP segments.

In the implemented configuration, the five segments considered most relevant (*top-5*) are retrieved together with their associated document metadata. The search results are then aggregated to construct the context provided to the language model, including the textual content of the retrieved segments and information about their corresponding articles, headings, source pages, and similarity scores.

The original question and the constructed context are then sent to the AMALIA-9B model, executed through Ollama. The model is instructed to generate the response based on the information provided in the context and to identify the relevant articles. Finally, the *workflow* formats the result and presents the generated response together with the corresponding documentary sources.

Figure 3 presents the overall workflow of this phase, from receiving the question to presenting the generated response and the retrieved documentary sources.

4 Environment Setup

The implementation was designed to be reproducible in a local environment. Running the solution requires Git, Python 3.10 or later, Jupyter, Docker, and Ollama. The main Python dependencies include *pymupdf*, *numpy*, *pandas*, *requests*, and *qdrant-client*.

The first step is to clone the two repositories that make up the solution:

```
git clone https://github.com/ruialexrib/legal-rag-pt.git
git clone https://github.com/ruialexrib/legal-rag-pt-n8n.git
```

The *legal-rag-pt* repository contains the components associated with ingestion, indexing, search, and evaluation, while *legal-rag-pt-n8n* contains the conversational application and the orchestration *workflow*.

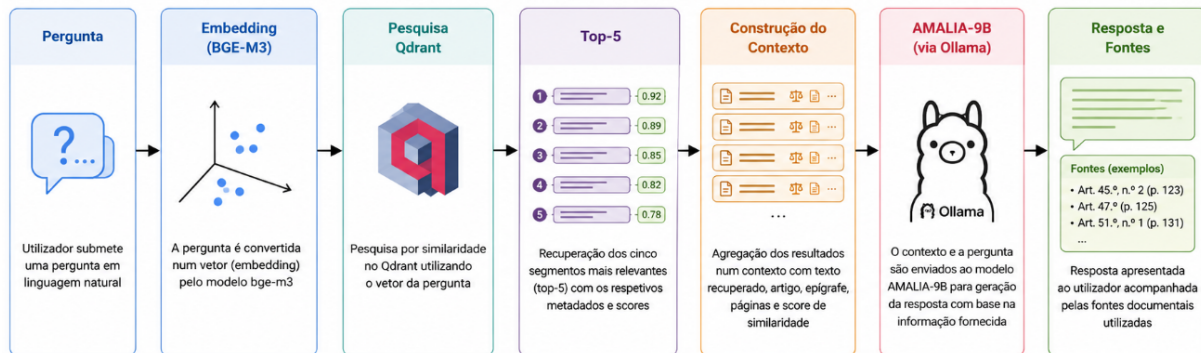


Figure 3: Query and response generation workflow. The question is converted into a vector representation using the **bge-m3** model and used to retrieve the five most relevant segments from Qdrant. The retrieved information is used to construct the context provided to the AMALIA-9B model, which generates the final response together with the corresponding documentary sources.

4.1 Ingestion and Indexing Environment

To prepare the environment for the first project, the corresponding directory must be accessed and a Python virtual environment created:

```
cd legal-rag-pt
python -m venv .venv
source .venv/bin/activate
pip install jupyter pymupdf numpy pandas requests qdrant-client
```

The model used to generate the *embeddings* must then be made available in Ollama:

```
ollama pull bge-m3
```

Qdrant is deployed locally through a Docker container. The `docker-compose.yml` file included in the repository defines the configuration required to start the vector database:

```
docker compose up -d
```

This command starts the Qdrant container in the background, exposing ports 6333 and 6334 and using persistent storage to preserve the vector collection data across restarts. The Qdrant REST API is available locally at: <http://localhost:6333>

After the environment has been prepared, the project *notebooks* should be executed sequentially, since each stage uses the outputs produced by the previous stages. Executing the ingestion and indexing steps creates and populates the `crmp_bge_m3` collection in Qdrant, which is required by the conversational application.

4.2 Conversational Application Environment

The conversational application is available in the `legal-rag-pt-n8n` repository. Before starting this environment, Qdrant must be running and the `crmp_bge_m3` collection must have been previously created and populated.

A language model suitable for answer generation must also be available in Ollama. In the implementation presented in this work, AMALIA-9B was used. Given the computational resources available in the development environment, namely an NVIDIA GeForce RTX 4050 GPU with 6 GB of memory, a pre-quantised version of the model in Q3_K_M format was selected. Quantisation reduces the memory required to execute the model, making it possible to run it on hardware with more limited resources.

```
ollama pull hf.co/ruialexrib/AMALIA-9B-0626-SFT-GGUF:Q3_K_M
```

This choice is not a requirement of the proposed architecture. The generative model may be replaced by another model compatible with Ollama, or by another quantisation variant, depending on the available computational resources. In such cases, the corresponding configuration in the *n8n workflow* must be updated.

The availability of the installed models can be checked using:

```
ollama list
```

The second environment can then be started by accessing the corresponding repository:

```
cd ../legal-rag-pt-n8n
docker compose up -d
```

In this case, Docker Compose starts the *n8n* container and exposes the interface on port 5678. The *n8n* interface is available locally at: <http://localhost:5678>

The *workflow* used by the solution is stored in:

```
workflows/legal-rag-pt-n8n.json
```

This file must be imported through the *n8n* interface using the *Import from File* option. After the import, the configuration of the different nodes should be reviewed, and the *workflow* should then be saved and activated.

5 Corpus Construction

Corpus construction transforms the original CRMP document into a structured collection of text segments suitable for subsequent *embedding* generation and vector indexing. The process is performed sequentially through four stages: document extraction, text normalisation, identification of the legal structure, and article segmentation.

5.1 Extraction and Normalisation

The first stage, implemented in the *01_pdf_extraction.ipynb notebook*, uses PyMuPDF to process the 662 pages of the original document and extract their textual content. Extraction is performed page by page, preserving the corresponding source page for each record. This approach maintains document provenance from the beginning of the processing pipeline and allows this information to be subsequently used when presenting the sources associated with the generated responses. The output of this stage is stored in *crmp_extracted.json*.

Text extracted directly from a PDF document may contain elements resulting from its formatting, such as consecutive spaces, line breaks, or other editorial artefacts. For this reason, the *02_text_preprocessing.ipynb notebook* performs a normalisation stage before the legal structure of the document is analysed.

The adopted normalisation process is deliberately conservative. Elements that do not contribute to the meaning of the content are corrected, while punctuation, numbering, and other elements relevant to interpretation and subsequent identification of the document structure are preserved. The result is stored in *crmp_cleaned.json*.

5.2 Document Structure

After normalisation, the *03_document_parsing.ipynb notebook* analyses the content to identify individual articles and reconstruct their corresponding document structure. For each article, information such as its heading, position within the document hierarchy, and corresponding pages is preserved.

This processing makes it possible to use the article as the legal unit of reference, rather than treating the document exclusively as a continuous sequence of text. This is particularly

relevant for retrieval, as it allows the subsequently retrieved segments to be related to the original structure of the CRMP and enables interpretable references to be presented to the user.

A total of 1,440 articles were identified and stored in `crmp_articles.json`. However, the structural analysis rules were developed specifically according to the organisation of the CRMP. Applying the same process to other documents may therefore require adapting the rules used to identify articles and other elements of their document structure.

5.3 Overlapping Chunking

Although an article represents a relevant document unit, some articles may be too long to be used directly as retrieval units. For this reason, the `04_document_chunking.ipynb notebook` segments longer articles into smaller text units (*chunks*).

Articles that do not exceed the defined size limit are kept intact. Longer articles are divided into segments containing a maximum of 350 words. An overlap of 50 words is maintained between consecutive segments to reduce the potential loss of context at segmentation boundaries.

Considering a maximum segment length $L = 350$ and an overlap $O = 50$, the stride between two consecutive segments is given by:

$$S = L - O = 350 - 50 = 300 \text{ words.} \quad (1)$$

Therefore, the final 50 words of one segment may also appear at the beginning of the following segment. This strategy aims to preserve information that might otherwise be artificially separated by a *chunk* boundary, while maintaining segments of a controlled size.

Each *chunk* retains the metadata required to establish its relationship with the original document, namely `article`, `article_title`, `page_start`, `page_end`, `chunk_index`, `num_chunks`, and `text`. Additional validation checks are performed to ensure unique identifiers, the absence of empty segments, and compliance with the defined segmentation limits.

At the end of this process, 1,617 *chunks* are obtained and stored in `crmp_chunks.json`. These segments constitute the document units used in the subsequent stage for *embedding* generation and vector index construction.

6 Embeddings and Indexing

After corpus construction, each *chunk* is converted into a vector representation and subsequently stored in the vector database. This stage establishes the representation used by the semantic retrieval mechanism, allowing submitted questions to be compared with the previously processed document segments.

6.1 Embedding Generation

The generation of *embeddings* is performed in the `05_embedding_generation.ipynb notebook`. For each of the 1,617 *chunks*, the corresponding textual content is sent to the local `/api/embed` endpoint provided by Ollama. The `bge-m3` model [1] transforms each segment into a 1,024-dimensional vector representation.

Using the same model to represent both document segments and, subsequently, user questions allows both types of content to be projected into the same vector space. As a result, during query processing, the representation of a question can be directly compared with the representations previously computed for the CRMP segments.

In the execution performed, *embeddings* were generated for all 1,617 segments, with an average processing time of approximately 0.84 seconds per segment, corresponding to a throughput of 1.19 segments per second. These values characterise the experimental environment and should

not be interpreted as general performance measures, since they depend, among other factors, on the available computational resources.

The generated vectors are stored in the `crmp_bge_m3_embeddings.json` file. Separating *embedding* generation from indexing allows the previously computed representations to be reused, avoiding the need to regenerate them whenever the vector collection must be rebuilt or reloaded.

6.2 Qdrant Indexing

The vector representations are indexed through the `06_qdrant_setup.ipynb notebook`, which creates the `crmp_bge_m3` collection in Qdrant. The collection is configured to store the 1,024-dimensional vectors produced by the `bge-m3` model and to use cosine similarity as the proximity measure.

Cosine similarity measures the proximity between two vectors based on the angle between them and can be expressed as:

$$\text{sim}_{\cos}(\mathbf{q}, \mathbf{x}) = \frac{\mathbf{q} \cdot \mathbf{x}}{\|\mathbf{q}\| \|\mathbf{x}\|}, \quad (2)$$

where $\mathbf{q}$ represents the vector corresponding to the question and $\mathbf{x}$ the vector associated with a document segment. Higher similarity values indicate greater proximity between the respective vector representations.

Each point stored in Qdrant associates the segment *embedding* with a *payload* containing its textual content and document metadata, including the article, heading, source pages, and information related to the *chunk*. This organisation allows a vector search to return not only the segments considered most relevant, but also the information required to identify their documentary provenance.

Once indexing is completed, the `crmp_bge_m3` collection constitutes the vector index used by the retrieval component. At query time, the *embedding* of a question is compared with the vectors in this collection, and the segments with the highest similarity are used to build the context subsequently provided to the language model.

7 Search and Evaluation

After constructing the vector index, the information retrieval process was implemented and evaluated. This evaluation focuses exclusively on the retrieval component of the RAG system, aiming to verify whether the submitted questions are able to identify the CRMP articles previously defined as relevant.

The quality of the responses produced by the generative model is not evaluated at this stage.

7.1 Vector Search

The search process is implemented in the `07_vector_search.ipynb notebook`. For each question, an *embedding* is generated using the same `bge-m3` model used during corpus indexing. The resulting vector is sent to Qdrant and compared with the representations of the segments previously stored in the `crmp_bge_m3` collection.

Using the same model during both indexing and query processing is necessary to ensure that questions and document segments are represented in the same vector space. Consequently, the model configuration and vector dimensionality must remain consistent across both stages.

The search returns the segments ordered according to their similarity to the question. For each result, the metadata associated with the segment is also retrieved, allowing the corresponding article, heading, and source pages in the original document to be identified.

7.2 Retrieval Evaluation

The evaluation is implemented in the `08_retrieval_evaluation.ipynb notebook` and uses a preliminary set of five questions for which the relevant CRMP articles were manually defined. For each question, the vector search is executed and the retrieved results are compared with the reference articles.

Given the limited size of this test set, the results should be interpreted only as an initial validation of the retrieval component. A more rigorous evaluation should use a substantially larger number of questions, covering different thematic areas of the CRMP, alternative formulations, different levels of complexity, and cases in which the relevant information may not be directly available in the corpus.

Since a single article may be divided into several *chunks*, different retrieved segments may correspond to the same article. To prevent these occurrences from being counted as independent results, the segment *ranking* is converted into an ordered list of distinct articles before the evaluation metrics are calculated.

The evaluation uses *Hit@K*, *Recall@K*, and *Mean Reciprocal Rank* (MRR), which measure different aspects of retrieval quality.

The *Hit@K* metric simply checks whether at least one relevant article appears among the first K results. For example, *Hit@5* is equal to 1 if at least one relevant article appears among the first five results, and 0 otherwise. For a question q , the metric is defined as:

$$\text{Hit@K}(q) = \begin{cases} 1, & \text{if } R_q \cap A_{q,K} \neq \emptyset, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

where R_q represents the set of articles considered relevant to question q , and $A_{q,K}$ represents the set of articles retrieved within the first K positions.

The *Recall@K* metric evaluates how many of the relevant articles are found among the first K results. For example, if a question has two relevant articles and only one appears among the first five results, the *Recall@5* value is 0.5. The metric is calculated as:

$$\text{Recall@K}(q) = \frac{|R_q \cap A_{q,K}|}{|R_q|}. \quad (4)$$

The *Mean Reciprocal Rank* (MRR), in turn, considers the position at which the first relevant result appears. The closer this result is to the top of the *ranking*, the higher the resulting score. For example, if the first relevant article appears in the first position, the score for that question is 1; if it appears in the second position, the score is 1/2; and if it appears in the third position, the score is 1/3. MRR corresponds to the average of these values across all evaluated questions:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (5)$$

where rank_q corresponds to the position of the first relevant article retrieved for question q .

7.3 Results

Table 1 presents the results obtained in the preliminary evaluation of the retrieval component.

In all five evaluated cases, a relevant article was retrieved in the first position, resulting in *Hit@1*, *Recall@1*, and MRR values of 1.000. Consequently, the same *Hit@K* and *Recall@K* values were obtained for the remaining values of K considered. The average observed search latency was 0.909 seconds.

These results show that, for the set of questions used, the retrieval mechanism successfully identified the reference articles and ranked them at the top of the results. However, they constitute only a preliminary validation of the implementation and do not allow the overall performance of the system to be generalised.

Table 1: Results of the preliminary semantic retrieval evaluation.

Metric	Result
Number of questions	5
Hit@1	1.000
Recall@1	1.000
Hit@3	1.000
Recall@3	1.000
Hit@5	1.000
Recall@5	1.000
Hit@10	1.000
Recall@10	1.000
MRR	1.000
Average latency	0.909 s

8 Conversational Application

The conversational component of the solution is implemented through an *n8n workflow*, which is responsible for coordinating the different operations performed from the moment a question is received until the final response is presented. This approach makes the processing flow visually explicit and allows the data produced at each stage to be inspected individually.

8.1 Service Configuration

n8n is executed in a Docker container using the Docker Compose configuration provided in the `legal-rag-pt-n8n` repository. The service is exposed on port 5678 and uses the persistent volume `n8n_data`, allowing workflows and application settings to be preserved across restarts.

After preparing the services described in Section 4, the *n8n* interface can be accessed at <http://localhost:5678>. The application *workflow*, provided in the file `workflows/legal-rag-pt-n8n.json`, is then imported into *n8n*.

8.2 Query Workflow

The *workflow* sequentially implements the operations required for the query process. Table 2 presents the main nodes and their respective responsibilities.

Table 2: Main nodes of the *n8n workflow* and their respective functions.

Node	Responsibility
Chat Trigger	Receive the question through the conversational interface.
Generate Embedding	Generate the vector representation of the question through Ollama.
Qdrant Search	Search for and retrieve the five most relevant <i>chunks</i> .
Build Context	Construct the context from the retrieved content and meta-data.
Generate Answer	Send the question and context to the generative model.
Format Response	Format the response and append the corresponding documentary sources.

After receiving the question, the **Generate Embedding** node requests its vector representation from Ollama. The resulting vector is then used by the **Qdrant Search** node to query the

`crmp_bge_m3` collection, retrieving the five *chunks* with the highest similarity scores together with their respective *payloads*.

The **Build Context** node transforms the retrieved results into a textual structure suitable for the generative model while preserving the content of the segments and their provenance information, namely article, heading, pages, and similarity score. Keeping these operations in separate nodes facilitates inspection of intermediate results and helps identify potential issues at each stage of the process.

8.3 Response Generation

After the context has been constructed, the original question and the retrieved segments are sent by the **Generate Answer** node to the language model executed through Ollama. In the evaluated implementation, the previously described quantised version of AMALIA-9B was used.

The system message provided to the model defines a set of instructions aimed at restricting generation to the retrieved content. In particular, the model is instructed to:

- answer exclusively on the basis of the provided context;
- not use external knowledge to fill missing information;
- indicate when the retrieved context does not contain sufficient information;
- answer in European Portuguese;
- identify the relevant articles whenever they are present in the context.

Finally, the **Format Response** node organises the result presented in the conversational interface and appends the retrieved sources, including the corresponding articles, headings, and source pages.

Including these references allows the user to identify and consult the documentary provenance of the retrieved information. However, neither these instructions nor the presentation of sources guarantee that every statement generated by the model is fully supported by the retrieved context. Therefore, the generated responses should be considered a support mechanism for document consultation and should be verified, whenever necessary, against the official CRMP text.

9 Conclusions and Future Work

This work presented the implementation of a local *Retrieval-Augmented Generation* architecture for querying the Regulatory Code of the Municipality of Porto using natural language. The solution integrates document processing and structuring, *embedding* generation and indexing, semantic retrieval, and response generation through a conversational interface. Preserving articles, headings, and source pages throughout the processing pipeline allows the retrieved results to be associated with their corresponding documentary provenance.

In the preliminary evaluation of the retrieval component, a relevant article was retrieved in the first position for all five evaluated questions, resulting in *Hit@1*, *Recall@1*, and MRR values of 1.000. These results confirm the correct operation of the retrieval mechanism for the evaluated cases. However, the limited number of questions does not allow the overall performance or robustness of the system to be generalised.

Future work should focus on expanding and diversifying the set of questions used for evaluation and on assessing the quality of the generated responses, considering aspects such as faithfulness to the retrieved context, completeness, and the accuracy of the provided references. Regarding retrieval, hybrid strategies combining semantic and lexical search, particularly through BM25, should be explored, together with *reranking* mechanisms and strategies to reduce redundancy between *chunks* belonging to the same article.

Further development should also consider transforming the current *notebooks* into reusable modules, introducing automated tests, and strengthening the security mechanisms required for potential deployment in a production environment.

References

- [1] Jianlv Chen et al. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. 2024.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics, 2019.
- [3] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [4] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [5] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [6] Vinícius Teles de Oliveira, Deivison Oliveira da Silva, Mateus de Almeida Souza, Maurício Rodrigues Lima, Sávio Salvarino Teles de Oliveira, and Thierson Couto Rosa. Retrieval-augmented generation and knowledge graphs in Portuguese-language legal documents. In Marlo Souza, Iria de Dios-Flores, Diana Santos, Larissa Freitas, Jackson Wilke da Cruz Souza, and Eugénio Ribeiro, editors, *Proceedings of the 17th International Conference on Computational Processing of Portuguese (PROPOR 2026) - Vol. 1*, pages 1–10, Salvador, Brazil, April 2026. Association for Computational Linguistics.
- [7] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics, 2014.
- [8] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [9] Afonso Simplício, Gonçalo Vinagre, Miguel Moura Ramos, Diogo Tavares, Rafael Ferreira, Giuseppe Attanasio, Duarte M. Alves, Inês Calvo, Inês Vieira, Rui Guerra, James Furtado, Beatriz Canaverde, Iago Paulo, Vasco Ramos, Diogo Glória-Silva, Miguel Faria, Marcos Treviso, Daniel Gomes, Pedro Gomes, David Semedo, André Martins, and João Magalhães. AMALIA Technical Report: A Fully Open Source Large Language Model for European Portuguese. *arXiv preprint arXiv:2603.26511*, 2026.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.